Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Intelligent Multi-Agent and Expert Systems (AI401), Semester VII

Instructions

1. Save your file as LAB_No_Roll_No.doc (e.g. LAB_4_U21AI001.doc).
2. Paste a snapshot of each step followed along with the result(s).
3. Record your observations and comments after the results.

Experiment 4

Reasoning Techniques — Inference Rules: Modus Ponens and Modus Tollens

Aim

To apply the sound inference rules Modus Ponens and Modus Tollens to a propositional knowledge base of implications, to derive the set of facts entailed by a given set of observations, and to distinguish these valid patterns from the fallacies of affirming the consequent and denying the antecedent.

Theory

Experiment 1 showed how knowledge is encoded; encoding alone derives nothing. An inference rule is a syntactic pattern that licenses writing down a new sentence when sentences matching its premises are already present. A rule is sound when every interpretation that satisfies the premises also satisfies the conclusion, so a sound rule can never turn true knowledge into a false conclusion. Soundness is a property of the pattern alone, which is why an inference engine can be written once and reused over any knowledge base.

Modus Ponens takes an implication $P \rightarrow Q$ together with the fact P and concludes Q. Modus Tollens takes the same implication together with the negated consequent $\neg Q$ and concludes $\neg P$. Both follow from the truth table of implication: $P \rightarrow Q$ is false only in the row where P is true and Q is false, so that row is eliminated by either premise pair. Modus Tollens is equivalent to Modus Ponens applied to the contrapositive $\neg Q \rightarrow \neg P$, and this equivalence is what an engine actually implements — it stores each rule once and matches it in either direction.

The two converse patterns are unsound and must be rejected. Affirming the consequent infers P from $P \rightarrow Q$ and Q; denying the antecedent infers $\neg Q$ from $P \rightarrow Q$ and $\neg P$. Both fail because an implication permits several distinct antecedents to produce the same consequent, so the consequent does not identify its cause. A rule with a conjunctive antecedent weakens Modus Tollens in a related way: from P_1 and $P_2 \rightarrow Q$ together with $\neg Q$ one may conclude only that at least one antecedent is false, which is a disjunction rather than a definite literal, and a definite literal is obtained only when all but one antecedent is independently known to be true.

Exhaustive application of Modus Ponens until no new fact appears computes the deductive closure of the knowledge base; this fixed-point procedure is the engine of Experiment 5 on forward chaining, while the goal-directed use of the same rules read backwards underlies backward chaining. Because both rules are monotonic, a derived fact is never withdrawn when new facts arrive — in contrast with the default reasoning with exceptions seen in Experiment 1 — and deriving both p and $\neg p$ therefore signals an inconsistent knowledge base rather than a revisable default.

Algorithm

Algorithm 1 — Deriving the closure by Modus Ponens

1. Represent every fact as a literal, a pair of a symbol and a polarity, so that p and ¬p are distinct entries of the fact set; represent every rule as a list of antecedent literals together with one consequent literal.
2. Scan the rule base once: for each rule whose antecedent literals are all present in the fact set, add the consequent literal to the fact set and record the rule identifier and the premises used as its justification.
3. Repeat step 2 until a complete scan adds no new literal; the fact set has then reached its fixed point and holds the deductive closure of the input facts.
4. Report each derived literal with the chain of rules that produced it, and report the knowledge base as inconsistent if the fact set contains a symbol together with its negation.

Algorithm 2 — Reasoning from negative evidence by Modus Tollens

1. For every rule, form its contrapositive by negating the consequent as the new premise and negating the antecedent as the new conclusion.
2. Given an observed negative literal ¬Q, select the rules whose consequent is Q; if such a rule has a single antecedent P, add ¬P to the fact set, and if it has several antecedents, add the definite literal only when all but one antecedent are already known true, otherwise record the disjunction of negated antecedents as a pending conclusion.
3. Feed every literal derived in step 2 back into Algorithm 1, since a new negative fact may satisfy the antecedent of a further rule, and alternate the two algorithms until neither adds anything.
4. Reject any query that matches affirming the consequent or denying the antecedent by reporting that the conclusion is not entailed, and give the interpretation that satisfies the premises while falsifying the conclusion as the counter-example.

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the inference engine below, including the rule base and the input facts, the literals derived by Modus Ponens with the justification of each, the literal derived by Modus Tollens from the negative observation, and the engine’s response to one query of each unsound pattern.

Observations

Comment on why the contrapositive is needed to apply Modus Tollens, on what can and cannot be concluded when the rule matched by Modus Tollens has more than one antecedent, on the counter-example that shows affirming the consequent to be unsound, and on why the fixed point reached in Algorithm 1 is sound but not complete for full propositional logic.

Exercises

1. Extend the rule base with a rule having three antecedent literals and apply Modus Tollens to its negated consequent. State the conclusion the engine can record at that point, then add facts one at a time and identify exactly the fact that turns the pending disjunction into a definite literal.
2. Add rules and facts that derive both a symbol and its negation. Extend the engine so that it detects the contradiction, prints the two justification chains that meet, and reports the smallest set of input facts whose removal restores consistency.